Light Intensity Measurement and Validation of the Inverse-Square and Lambert's Cosine Laws

Azhari Abbas

Objective

The purpose of this experiment was to test the Inverse-Square Law of radiation intensity and Lambert's Cosine Law of angular dependence with an Arduino light sensor. The intention was to measure light intensity at varying distances and angles of incidence while comparing expected values with experimentally derived results.

Methodology

An Arduino Uno equipped with a photoresistor (LDR) circuit was used to record light intensity in analog-to-digital converter (ADC) units. The circuit functioned as a voltage divider, where the LDR's resistance changed in response to the light intensity. The setup consisted of:

- One leg of the LDR connected to the 5 V pin and the other leg connected to the A0 analog input.
- A 10 kΩ resistor connected between the A0 pin and GND.

This configuration produced a measurable voltage at A0 proportional to the illumination level. The physical setup is shown in Figure 1.

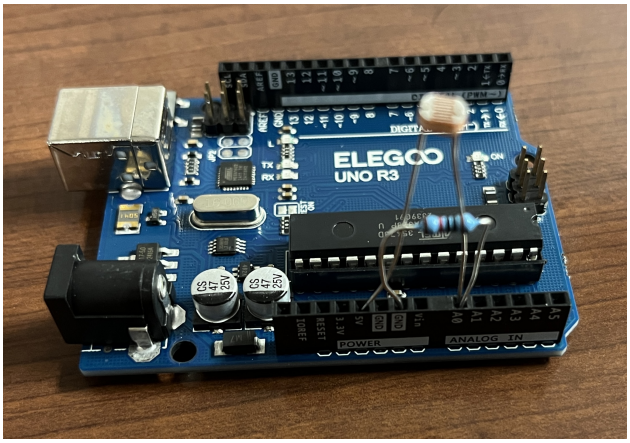

Figure 1: Arduino Uno with LDR and resistor voltage divider used for light intensity measurements.

Two experimental runs were performed:

1. Intensity vs. distance from 1 ft to 20 ft, maintaining the sensor normal to the light source.
2. Intensity vs. incidence angle from -90° to $+90^\circ$ at a fixed distance.

Theoretical fits were applied using:

$$I(r) = \frac{a}{r^2}, \qquad I(\alpha) = b\cos(\alpha) + c$$

where r is the distance and α is the incidence angle.

Results

Table 1: Measured intensity vs. distance from the light source.

Distance (ft)	**Intensity (ADC units)**
1	711
2	609
3	520
4	472
5	413
6	353
7	305
8	263
9	209
10	167
11	143
12	116
13	109
14	98
15	85
16	67
17	61
18	57
19	41
20	30

Table 2: Measured intensity vs. incidence angle at fixed distance.

Angle (°)	**Intensity (ADC units)**
-90	556
-75	586
-60	621
-45	657
-30	667
-15	674
0	687
15	666
30	654
45	610
60	602
75	580
90	564

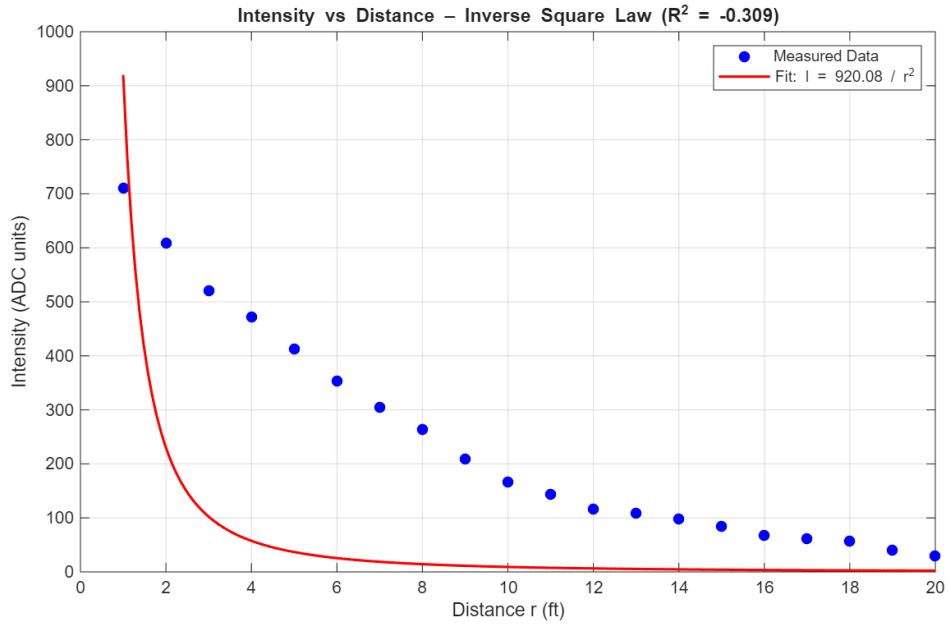

Figure 2: Measured intensity vs. distance and best-fit inverse-square model.

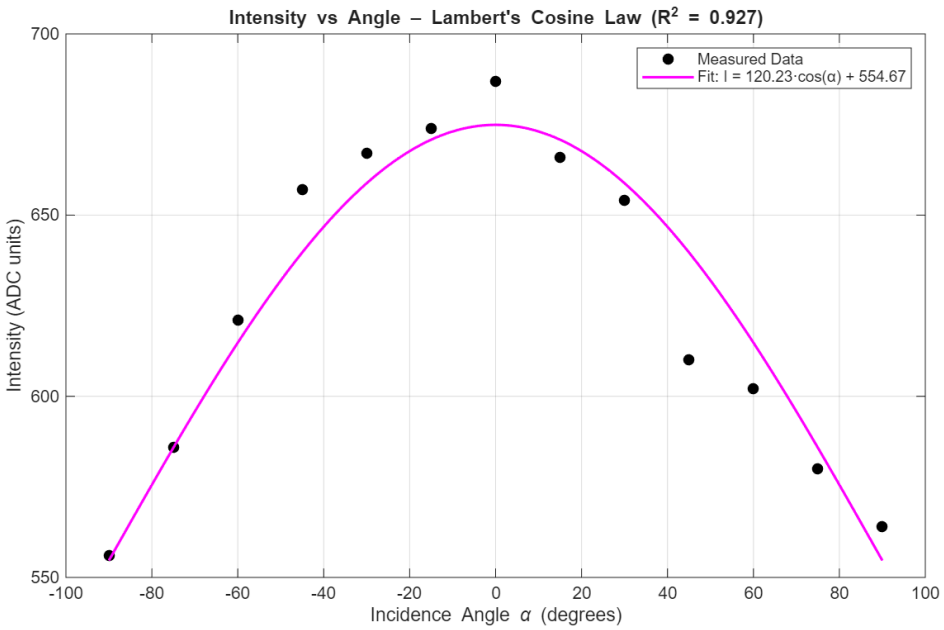

Figure 3: Measured intensity vs. incidence angle and best-fit cosine model.

The inverse-square fit yielded $R^2 = -0.309$, while the cosine fit yielded $R^2 = 0.927$.

Discussion

The readings for angular deviation are closely aligned with Lambert's Cosine Law since R^2 is 0.927. This confirms that the photoresistor output is proportional to the effective incident light flux. As the angle of incidence increased, the effective area of illumination on the sensor decreased, resulting in a standard cosine variance of intensity.

On the other hand, the negative R^2 value indicates that the measured data did not follow the expected $1/r^2$ trend. Several factors likely contributed to this issue, including ambient light interference and alignment errors, because maintaining perfect perpendicularity and consistent positioning over longer distances was challenging. Also, the light source (iPhone flashlight) was not a perfect point emitter.

The overall behavior of the data still reflects the qualitative principles of the inverse-square law. The measured intensity decreased with increasing distance, but not at the theoretical rate.

Conclusion

The experiment qualitatively validates both the inverse-square and cosine relationships for radiative intensity, but high accuracy was achieved only for the angular dependence.

Appendix A: MATLAB Code

```
clc; clear; close all;

% Intensity vs Distance Data
r = (1:20)';            % distance in feet
I_r = [711 609 520 472 413 353 305 263 209 167 143 116 109 98 85 67
   61 57 41 30]';

% Fit model: I = a / r^2
model1 = fittype('a / (x^2)', 'independent', 'x', 'coefficients', 'a'
   );
fit1 = fit(r, I_r, model1);

% Predicted fit
r_fit = linspace(min(r), max(r), 200);
I_fit = fit1(r_fit);

% R
resid = I_r - fit1(r);
SSres = sum(resid.^2);
SStot = sum((I_r - mean(I_r)).^2);
R2_1 = 1 - SSres/SStot;

% Plot
figure(1)
plot(r, I_r, 'bo', 'MarkerFaceColor', 'b'); hold on
plot(r_fit, I_fit, 'r-', 'LineWidth', 1.5)
xlabel('Distance r (ft)')
ylabel('Intensity (ADC units)')
title(sprintf('Intensity vs Distance    Inverse Square Law (R^2 =
   %.3f)', R2_1))
legend('Measured Data', sprintf('Fit: I = %.2f / r^2', fit1.a), '
   Location', 'northeast')
grid on


% Intensity vs Angle Data
theta = [-90 -75 -60 -45 -30 -15 0 15 30 45 60 75 90]';
I_theta = [556 586 621 657 667 674 687 666 654 610 602 580 564]';

% Fit model: I = b * cosd(theta) + c
model2 = fittype('b*cosd(x) + c', 'independent', 'x', 'coefficients',
    {'b','c'});
fit2 = fit(theta, I_theta, model2);

% Predicted fit
```

```
theta_fit = linspace(-90, 90, 200);
I_theta_fit = fit2(theta_fit);

% R
resid2 = I_theta - fit2(theta);
SSres2 = sum(resid2.^2);
SStot2 = sum((I_theta - mean(I_theta)).^2);
R2_2 = 1 - SSres2/SStot2;

% Plot
figure(2)
plot(theta, I_theta, 'ko', 'MarkerFaceColor', 'k'); hold on
plot(theta_fit, I_theta_fit, 'm-', 'LineWidth', 1.5)
xlabel('Incidence Angle \alpha (degrees)')
ylabel('Intensity (ADC units)')
title(sprintf('Intensity vs Angle    Lambert''s Cosine Law (R^2 =
    %.3f)', R2_2))
legend('Measured Data', sprintf('Fit: I = %.2f cos(  ) + %.2f', fit2
    .b, fit2.c), 'Location', 'northeast')
grid on
```

Listing 1: MATLAB script for data fitting and plotting.

Appendix B: Arduino Code

```
const int LDR_PIN = A0;
const float VREF = 5.0;        // UNO analog reference
const int ADC_MAX = 1023;
const float R_FIXED = 10000.0; // 10k to GND

const int N_SAMPLES = 10;

void setup() {
  Serial.begin(115200);
  // CSV header
  Serial.println("ms,adc,voltage_V,R_ldr_ohm");
}

void loop() {
  // Average a few samples to reduce noise
  long sum = 0;
  for (int i = 0; i < N_SAMPLES; i++) {
    sum += analogRead(LDR_PIN);
    delay(2);
  }
  float adc = sum / float(N_SAMPLES);
```

```
22
23    // Convert to voltage
24    float voltage = (adc / ADC_MAX) * VREF;
25
26    // Compute LDR resistance from divider: Vout = 5 * (R_fixed / (
          R_LDR + R_fixed))
27    // => R_LDR = R_fixed * (5/Vout - 1)
28    float r_ldr = (voltage > 0.001) ? R_FIXED * (VREF / voltage - 1.0)
          : 1e9;
29
30    // Stream CSV: time(ms), adc, voltage, resistance
31    Serial.print(millis()); Serial.print(',');
32    Serial.print((int)adc); Serial.print(',');
33    Serial.print(voltage, 4); Serial.print(',');
34    Serial.println(r_ldr, 1);
35
36    delay(200); // ~5 Hz logging
37 }
```

Listing 2: Arduino sketch for LDR light intensity measurement.